Dubbele AES kraken met een meet-in-the-middle-aanval

Uitleg van de code, stap voor stap, met een werkwijze voor andere opdrachten

1. Waar gaat de opdracht over?

Een bedrijf zegt: "AES is goed. Wij gebruiken AES **twee keer** met twee sleutels van 20 bit. Dat is 2 x 20 = 40 bit, dus onkraakbaar." Dat klopt niet. In deze uitleg zie je waarom, en hoe je het zelf aanpakt.

Wat je hebt:

Wat	Uitleg
De promo-code (hex)	Het geheim dat je wilt lezen. Het is twee keer versleuteld.
Het bestand enc.bmp	Een versleutelde afbeelding. Dit gebruik je om de sleutels te vinden.
Het sleutelformaat	Een sleutel is 16 bytes: 3 bytes met een getal (0 tot 2^{20}) en 13 nulbytes.

Wat je moet doen: de twee sleutels K1 en K2 vinden en daarmee de promo-code ontsleutelen.

2. De theorie: waarom 2 x 20 bit geen 40 bit is

De dubbele versleuteling werkt zo:

```
platte tekst P  --(versleutel met K1)-->  middenwaarde M  --(versleutel met K2)-->  ciphertext C
```

De dom-methode (brute force): probeer alle combinaties van K1 en K2. Dat zijn 2^{20} x 2^{20} = 2^{40} combinaties, ongeveer 1.000.000.000.000. Dat is veel te langzaam.

De slimme methode (meet-in-the-middle): je hebt *een* blok waarvan je zowel de platte tekst P als de ciphertext C kent. Dan:

Stap	Wat je doet	Aantal pogingen
Van links	Versleutel P met elke mogelijke K1. Bewaar elke middenwaarde M in een opzoektabel.	2^{20} (ongeveer 1 miljoen)
Van rechts	Ontsleutel C met elke mogelijke K2. Kijk of de uitkomst in de tabel staat.	2^{20} (ongeveer 1 miljoen)
Match	Als een M aan beide kanten hetzelfde is, heb je K1 en K2 gevonden.	-

Totaal: 2^{20} + 2^{20} = ongeveer 2 miljoen pogingen in plaats van 1000 miljard. Dubbele AES is dus maar ongeveer zo sterk als **een** sleutel van 21 bit.

Belangrijk: zonder bekend paar (P, C) werkt dit niet. Je hebt dan niets om je gok mee te vergelijken. Het zoeken naar zo'n paar is dus het echte puzzelwerk. Dat staat in hoofdstuk 4.

3. De code, blok voor blok

Blok 1: bibliotheken en gegevens inlezen

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad
import struct

data = open("enc.bmp", "rb").read()
promo_hex = "df7c7dfb...."
promo = bytes.fromhex(promo_hex)
```

Regel	Uitleg
AES	De versleutelfunctie uit de bibliotheek pycryptodome.
unpad	Haalt de padding (opvulling) weg aan het einde van de ontsleutelde tekst.
struct	Zet getallen om naar bytes in een bepaalde volgorde. Dat heb je nodig voor de BMP-header.
open(..., "rb").read()	Leest het bestand als bytes ("rb" = read binary).
bytes.fromhex(...)	De promo-code staat als hex-tekst. Dit maakt er echte bytes van. Twee hex-tekens zijn een byte.

Blok 2: het bekende ciphertext-blok

```
known_ct = data[54:70]
```

Dit is een Python-slice: de bytes van positie 54 tot en met 69. Dat zijn precies 16 bytes, en dat is een AES-blok. Waarom 54? Omdat de versleuteling in deze afbeelding pas na 54 bytes begint. Hoe je dat zelf ziet, lees je in hoofdstuk 4.

Blok 3: de kandidaten voor de bekende platte tekst

```
candidates = {
    "BGRA": struct.pack("<4I", 0x00FF0000, 0x0000FF00, 0x000000FF, 0xFF000000),
    "RGBA": struct.pack("<4I", 0x000000FF, 0x0000FF00, 0x00FF0000, 0xFF000000),
    "ARGB-ish": struct.pack("<4I", 0xFF000000, 0x00FF0000, 0x0000FF00, 0x000000FF),
}
```

Hier gok je wat er in die eerste 16 bytes *stond* voordat het werd versleuteld. In een BMP met een V5-header staan op die plek vier kleurmaskers (rood, groen, blauw, alpha), elk 4 bytes. Bij een gewone 32-bit afbeelding zijn dat bekende waarden. Je hebt maar een paar mogelijke volgordes, dus je probeert ze allemaal.

Uitleg van `struct.pack("<4I", ...)`: < betekent little-endian (de kleinste byte eerst, zoals BMP dat doet). `4I` betekent vier getallen van 32 bit zonder teken. Samen geeft dat 16 bytes.

Blok 4: de sleutel maken

```
def make_key(k):
    return k.to_bytes(3, "big") + b"\x00" * 13
```

Dit volgt het sleutelformaat uit de opdracht: het getal k wordt 3 bytes (big-endian), daarna komen 13 nulbytes. Samen zijn dat 16 bytes, de lengte van een AES-128-sleutel.

Blok 5: de meet-in-the-middle-functie

```
def mitm(known_pt, known_ct):
    table = {}
    for k1 in range(2**20):
        mid = AES.new(make_key(k1), AES.MODE_ECB).encrypt(known_pt)
        table[mid] = k1
    for k2 in range(2**20):
        mid = AES.new(make_key(k2), AES.MODE_ECB).decrypt(known_ct)
        if mid in table:
            return table[mid], k2
    return None
```

Deel	Wat gebeurt er?
table = {}	Een lege dictionary (opzoektabel). De sleutel van de tabel is de middenwaarde, de waarde is k1.
Eerste for-loop	Voor elke k1: versleutel de bekende platte tekst en bewaar "middenwaarde -> k1". Dit is de kant van links.
Tweede for-loop	Voor elke k2: **ont**sleutel de bekende ciphertext. Dit is de kant van rechts.
if mid in table	Staat deze middenwaarde in de tabel? Dan zijn beide kanten gelijk. Je geeft (k1, k2) terug.
MODE_ECB	Je versleutelt maar een los blok. ECB is dan het eenvoudigst, want er is geen IV of teller.

Let op de richting: links **versleutel** je, rechts **ontsleutel** je. Als je dit omdraait, vind je nooit een match.

Blok 6: de kandidaten proberen

```
for name, pt in candidates.items():
    print("trying", name)
    res = mitm(pt, known_ct)
    if res:
        k1, k2 = res
        print("found:", k1, k2)
        break
else:
    raise SystemExit("no match, check your plaintext guess")
```

Je probeert elke gok na elkaar. Bij de eerste match stopt de loop (`break`). Als geen enkele gok werkt, geeft het `else`-deel een foutmelding. Dat betekent dat je gok over de platte tekst fout was.

Blok 7: de promo-code ontsleutelen

```
c1 = AES.new(make_key(k1), AES.MODE_ECB)
c2 = AES.new(make_key(k2), AES.MODE_ECB)

plain = c1.decrypt(c2.decrypt(promo))
try:
    plain = unpad(plain, 16)
except ValueError:
    pass
print(plain)
```

De versleuteling was eerst K1 en daarna K2. Om dat terug te draaien, ga je in **omgekeerde volgorde**: eerst K2 ontsleutelen, dan K1. Daarna haal je de padding weg met `unpad`. De `try/except` zorgt dat het programma niet crasht als er geen geldige padding is.

> Dit is hetzelfde principe als bij het aantrekken van sokken en schoenen: je trekt ze in omgekeerde volgorde weer uit.

Blok 8: de afbeelding ontsleutelen (controle)

```
body = c1.decrypt(c2.decrypt(data[54:]))
with open("dec.bmp", "wb") as f:
    f.write(data[:54] + body[:9300])
```

Hiermee ontsleutel je de hele afbeelding. De eerste 54 bytes waren niet versleuteld, dus die plak je er weer voor. Je knipt het resultaat af op 9300 bytes, omdat de rest padding is (9354 min 54 = 9300). Als je `dec.bmp` kunt openen en er een echte afbeelding ziet, weet je zeker dat de sleutels kloppen.

4. Hoe kom je zelf aan de bekende platte tekst?

Dit is het moeilijkste deel, en het is ook het deel dat je bij andere opdrachten opnieuw moet doen. Hier is de redenering die we hebben gevolgd.

Stap A: bekijk de ruwe bytes

```
data = open("enc.bmp", "rb").read()
print(len(data))
print(data[:64].hex(" "))
```

De uitvoer begon met `42 4d 8a 24 00 00 ...`. De bytes `42 4d` zijn de letters "BM". Dat is de handtekening van een BMP-bestand. De eerste bytes waren dus **niet** versleuteld.

Stap B: zoek waar de versleuteling begint

Aan het begin staan leesbare waarden: veel nullen en kleine getallen. Vanaf positie 54 zagen de bytes er willekeurig uit (`f4 3b ad 88 ...`). Daar begint dus de versleutelde data. Bestandsgrootte min 54 is 9312, en dat is precies 582 x 16. Dat klopt dus met AES-blokken van 16 bytes.

Stap C: bepaal de modus

```
from collections import Counter
enc = data[54:]
blocks = [enc[i:i+16] for i in range(0, len(enc), 16)]
print(Counter(blocks).most_common(3))
```

Als hetzelfde ciphertext-blok vaak terugkomt (hier 162 keer), dan geeft gelijke platte tekst gelijke ciphertext. Dat is het kenmerk van **ECB**. Daarom mag je een los blok versleutelen zonder IV. Let op: je moet de blokken vanaf positie 54 indelen, niet vanaf 0. Anders liggen je blokken verkeerd.

Stap D: lees de header

Positie	Bytes	Betekenis
0-1	42 4d	"BM" = BMP-handtekening
2-5	8a 24 00 00	Bestandsgrootte = 0x248a = 9354 (little-endian)
10-13	8a 00 00 00	De pixeldata begint op positie 138
14-17	7c 00 00 00	Grootte van de header = 124, dus een **BITMAPV5HEADER**
30-33	03 00 00 00	Compressie = 3 (BI_BITFIELDS): er zijn kleurmaskers

Een V5-header is 124 bytes en begint op positie 14. De eerste 40 bytes daarvan eindigen op positie 54. Vanaf 54 komen dus de **vier kleurmaskers** (rood, groen, blauw, alpha), samen precies 16 bytes. Dat is ons bekende blok. Je kunt de lay-out opzoeken door te zoeken op "BITMAPV5HEADER" in de Microsoft-documentatie.

Stap E: maak je gok

Voor een gewone 32-bit BGRA-afbeelding zijn de maskers 0x00FF0000, 0x0000FF00, 0x000000FF en 0xFF000000. Je zet ze om naar bytes met `struct.pack("<4I", ...)`. Omdat je niet zeker bent over de volgorde, probeer je een paar varianten. Dat kost niets extra's: elke

poging duurt maar enkele seconden of minuten.

Alternatief: de meest voorkomende blok

Je kunt ook het ciphertext-blok nemen dat het vaakst voorkomt (de vlakke achtergrond) en raden dat de platte tekst 16 nulbytes of 16 bytes `ff` is. Dit is eenvoudiger, maar het is een gok over de *inhoud*. De header-methode gebruikt het *formaat* van het bestand en is daarom betrouwbaarder.

5. Algemeen stappenplan voor andere opdrachten

Gebruik deze lijst als je een nieuwe crypto-opdracht krijgt. De volgorde is altijd hetzelfde.

Stap	Vraag die je jezelf stelt	Wat je doet
1	Welk algoritme en welke modus?	Lees de opdracht. Zoek herhaalde blokken (ECB) of een teller/IV (CBC, CTR).
2	Wat is het zwakke punt?	Hergebruikte nonce? Te kleine sleutel? Dubbele versleuteling? Zwakke modus?
3	Welke gegevens heb ik?	Maak een lijst: ciphertexts, bekende platte tekst, sleutelformaat.
4	Heb ik een bekend paar (P, C)?	Zo niet: zoek een bestandsformaat met een vaste header (BMP, PNG, PDF, ZIP) of een vaste tekst ("flag{").
5	Welke aanval past?	Zie de tabel hieronder.
6	Schrijf de code in kleine stappen	Eerst gegevens inlezen en printen, dan de aanval, dan de controle.
7	Controleer het resultaat	Is de padding geldig? Is de tekst leesbaar? Opent het bestand?

Welke aanval bij welk probleem?

Situatie	Aanval	Kernidee
Zelfde nonce in CTR (zoals oefening 1)	XOR van twee ciphertexts	keystream = C1 XOR P1, daarna P2 = C2 XOR keystream
Twee keer versleuteld met kleine sleutels	Meet-in-the-middle	Zoek van beide kanten naar dezelfde middenwaarde
Een kleine sleutel	Brute force	Probeer alle sleutels en controleer of de tekst leesbaar is
ECB-modus met afbeelding	Patronen herkennen	Gelijke blokken blijven gelijk, de vorm is soms nog zichtbaar

6. Veelgemaakte fouten

Fout	Gevolg	Oplossing
Blokken indelen vanaf positie 0	Alle blokken liggen verkeerd	Begin bij waar de versleuteling echt begint (hier 54)
Encrypt en decrypt omdraaien	Geen match	Links versleutelen, rechts ontsleutelen
Verkeerde byte-volgorde (endianness)	Je gok klopt niet	BMP gebruikt little-endian: < in struct
Ontsleutelen in de verkeerde volgorde	Onzin als uitvoer	Eerst de laatste sleutel (K2), dan K1
Hex niet omzetten	Foutmelding of onzin	Gebruik `bytes.fromhex()`
Padding vergeten	Extra tekens aan het einde	Gebruik `unpad(x, 16)`

7. Kleine woordenlijst

Begrip	Uitleg
Plaintext / platte tekst	De leesbare tekst voor het versleutelen.
Ciphertext	De versleutelde tekst.
Known-plaintext attack	Een aanval waarbij je een deel van de platte tekst kent.
ECB	Elk blok van 16 bytes wordt apart versleuteld. Gelijke blokken geven gelijke uitvoer.
Padding	Opvulling zodat de lengte een veelvoud van 16 is (PKCS7).
Little-endian	De kleinste byte staat vooraan. Het getal 0x248a wordt opgeslagen als 8a 24.
Dictionary	Een opzoektabel in Python: sleutel -> waarde.
Brute force	Alle mogelijkheden proberen.